

# 机器视觉技术

## 第4章：边缘检测

Edge Detection — 中文翻译版（含原图）

每页包含：原始英文幻灯片截图 + 中文翻译解读  
翻译时间：2026年6月 | 基于 Tesseract OCR + 人工校对

# Edge detection

## 边缘检测

本章介绍计算机视觉中的边缘检测技术，包括图像梯度计算、Canny边缘检测器等核心算法。边缘检测是机器视觉中最基础的特征提取步骤，为后续的分割、识别、跟踪等高层任务提供关键输入。

| Machine Vision Technology                                    |                            |                          |                    |                       |                       |                      |     |
|--------------------------------------------------------------|----------------------------|--------------------------|--------------------|-----------------------|-----------------------|----------------------|-----|
| Semantic information                                         |                            |                          |                    | Metric 3D information |                       |                      |     |
| Pixels                                                       | Segments                   | Images                   | Videos             | Camera                |                       | Multi-view Geometry  |     |
| Convolutions<br>Edges & Fitting<br>Local features<br>Texture | Segmentation<br>Clustering | Recognition<br>Detection | Motion<br>Tracking | Camera<br>Model       | Camera<br>Calibration | Epipolar<br>Geometry | SFM |
| 10                                                           | 4                          | 4                        | 2                  | 2                     | 2                     | 2                    | 2   |

## 机器视觉技术体系

本课程涵盖机器视觉的完整技术栈：

- 底层特征：边缘与拟合 ← 本章重点
- 局部特征：角点、斑点、纹理
- 中层处理：分割、聚类
- 高层任务：识别、检测、跟踪
- 几何视觉：相机模型、标定、对极几何、多视图

边缘检测是整个视觉处理流程的第一步，其质量直接影响后续所有处理。

## Edge detection

---

- **Goal:** Identify sudden changes (discontinuities) in an image
  - Intuitively, most semantic and shape information from the image can be encoded in the edges
  - More compact than pixels
- **Ideal:** artist's line drawing (but artist is also using object-level knowledge)



Source: D. Lowe

---

### 边缘检测简介

目标：识别图像中像素值的突然变化（不连续性）。

直觉理解：图像中大部分语义和形状信息可以编码在边缘中——比原始像素紧凑得多。

理想状态：像艺术家画的线条一样简洁，但艺术家还使用了物体层面的知识来选择性描绘。

核心优势：边缘大大压缩了数据量，同时保留了关键的形状和结构信息。

(图片来源：D. Lowe)

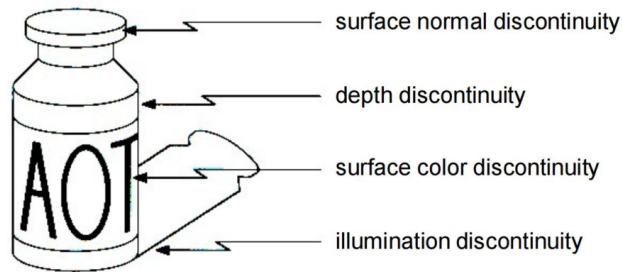
---



## Origin of edges

---

Edges are caused by a variety of factors:



Source: Steve Seitz

---

## 边缘的来源

边缘由多种物理因素引起：

- ①表面法向量不连续— 物体表面方向突变（如立方体的棱）
- ②深度不连续— 前景与背景之间的距离突变
- ③表面颜色不连续— 不同颜色或纹理区域的边界
- ④光照不连续— 阴影边界或高光区域

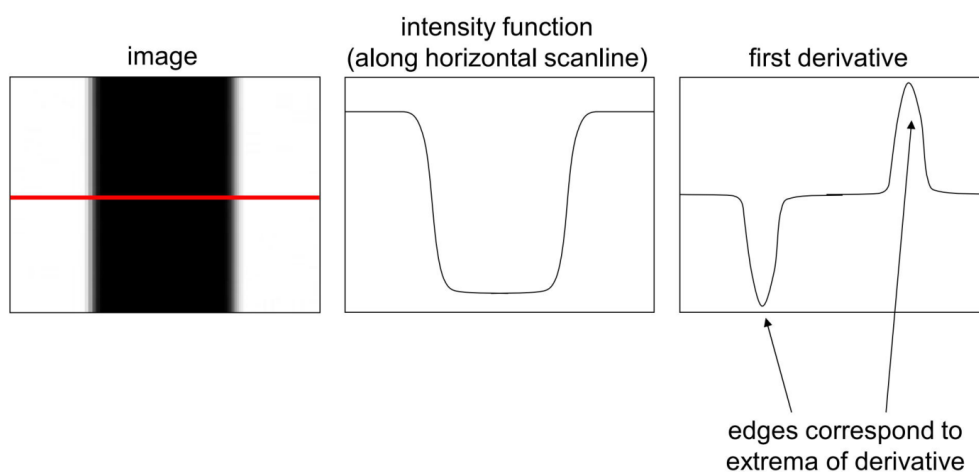
共同特征：所有这些不连续性在图像中都表现为亮度的快速变化。

(图片来源: Steve Seitz)

---

## Characterizing edges

- An edge is a place of rapid change in the image intensity function



## 边缘的数学特征

边缘 = 图像亮度函数中快速变化的位置。

考虑图像某一行沿水平方向的亮度函数  $f(x)$ :

- 原始信号在边缘处产生突变
- 一阶导数  $f'(x)$  在边缘处出现极值
- 通过寻找  $|f'(x)|$  的局部最大值来定位边缘

核心思想：边缘检测 → 计算图像导数 → 寻找极值点。

(图片来源: S. Lazebnik)

学习补充:

- 边缘本质上就是亮度函数发生快速变化的位置。
- 真实图像里通常先平滑再求导，否则噪声会把导数放大。

## Derivatives with convolution

For 2D function  $f(x, y)$ , the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

For discrete data, we can approximate using finite differences:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

To implement above as convolution, what would be the associated filter?

Source: K. Grauman

### 用卷积计算导数

对于二维函数  $f(x, y)$ ，偏导数的连续定义为：

$$\partial f / \partial x = \lim(\varepsilon \rightarrow 0) [f(x + \varepsilon, y) - f(x, y)] / \varepsilon$$

对于离散图像数据，使用有限差分近似：

$$\partial f(x, y) / \partial x \approx f(x + 1, y) - f(x, y)$$

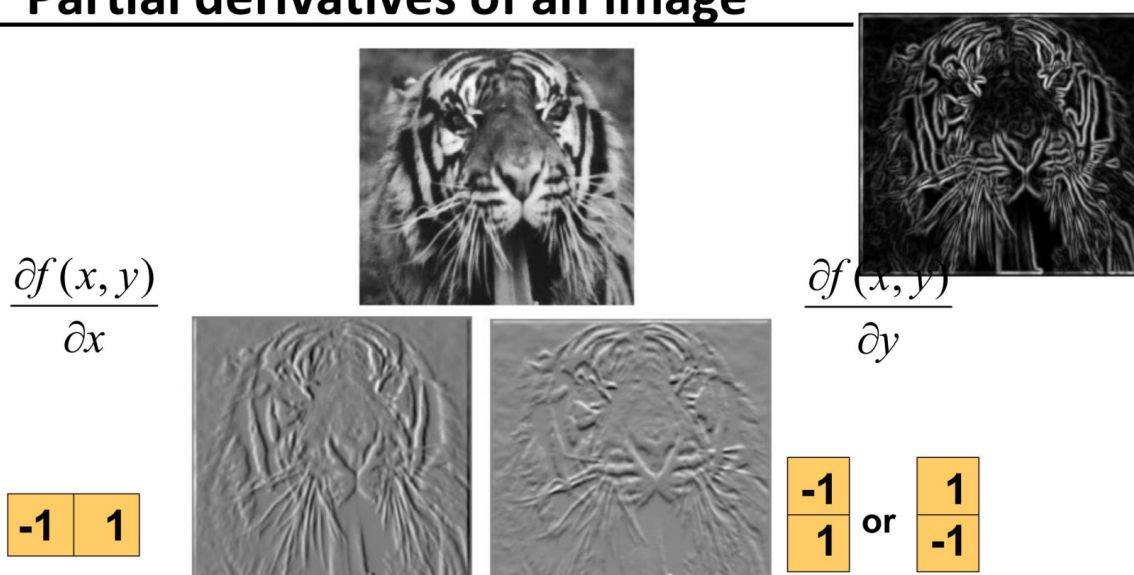
卷积实现：上述运算等价于用滤波器  $[-1, 1]$  对图像做卷积。

(图片来源：K. Grauman)

学习补充：

- 图像可以看成亮度函数，偏导数描述某个方向上的变化速度。
- x 方向导数更容易响应竖直边，y 方向导数更容易响应水平边。
- 梯度向量指向变化最快的方向，而边缘方向与梯度方向垂直。

## Partial derivatives of an image



Which shows changes with respect to x?

Source: S. Lazebnik

## 图像的偏导数

对图像分别沿 x 和 y 方向求偏导数：

- $\partial f / \partial x$ : 检测垂直边缘——图像在水平方向的变化
- $\partial f / \partial y$ : 检测水平边缘——图像在垂直方向的变化

观察偏导数图像即可判断：

→ 哪个滤波器响应垂直边缘？哪个响应水平边缘？

梯度方向与边缘方向垂直是边缘检测中的一个关键关系。

学习补充：

- 图像可以看成亮度函数，偏导数描述某个方向上的变化速度。
- x 方向导数更容易响应竖直边，y 方向导数更容易响应水平边。
- 梯度向量指向变化最快的方向，而边缘方向与梯度方向垂直。

## Finite difference filters

Other approximations of derivative filters exist:

$$\text{Prewitt: } M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

$$\text{Sobel: } M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

$$\text{Roberts: } M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Source: K. Grauman

## 常用导数近似算子

Prewitt 算子 (3×3) :

$$M_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Sobel 算子 (3×3, 中心加权) :

$$M_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix}$$

Roberts 算子 (2×2, 对角线) :

$$M_x = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \quad M_y = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

Sobel 最常用——在中心位置赋予更高权重，对噪声更鲁棒。

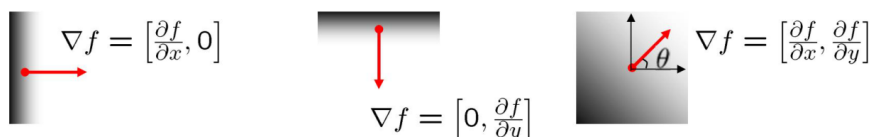
学习补充：

- Prewitt、Sobel、Roberts 都是在用离散差分近似导数。
- Sobel 在中间行/列加了更高权重，所以比 Prewitt 更抗噪；Roberts 用  $2 \times 2$  对角差分，响应更敏感。

(图片来源: K. Grauman)

## Image gradient

The gradient of an image:  $\nabla f = \left[ \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity

- How does this direction relate to the direction of the edge?

The gradient direction is given by  $\theta = \tan^{-1} \left( \frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Source: S. Seitz

## 图像梯度

图像梯度定义为偏导数向量：

$$\nabla f = [\partial f / \partial x, \partial f / \partial y]$$

梯度方向：指向亮度增加最快的方向。

梯度与边缘的关系：梯度方向  $\perp$  边缘方向。

梯度方向角：  $\theta = \tan^{-1}((\partial f / \partial y) / (\partial f / \partial x))$

梯度幅值（边缘强度）：

$$\|\nabla f\| = \sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2}$$

幅值越大  $\rightarrow$  该位置越可能是边缘。

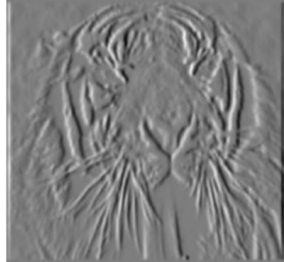
(图片来源：S. Seitz)

学习补充：

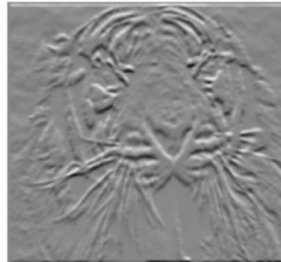
- 图像可以看成亮度函数，偏导数描述某个方向上的变化速度。
- x 方向导数更容易响应竖直边，y 方向导数更容易响应水平边。
- 梯度向量指向变化最快的方向，而边缘方向与梯度方向垂直。

## Gradient Magnitude

---



X-Derivative



Y-Derivative



Gradient Magnitude

---

### 梯度幅值可视化

分别计算两个方向的导数后合成梯度幅值：

- X方向导数图：突出垂直方向的边缘
- Y方向导数图：突出水平方向的边缘
- 梯度幅值图：综合所有方向的边缘强度

梯度幅值 = 每个像素位置的"边缘可能性"——亮的像素对应强边缘。

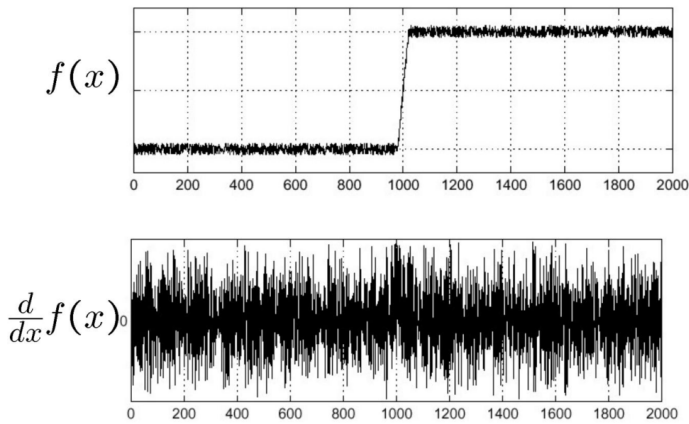
学习补充：

- 图像可以看成亮度函数，偏导数描述某个方向上的变化速度。
  - x 方向导数更容易响应竖直边，y 方向导数更容易响应水平边。
  - 梯度向量指向变化最快的方向，而边缘方向与梯度方向垂直。
-

## Effects of noise

Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal



Where is the edge?

Source: S. Seitz

### 噪声对边缘检测的影响

取图像的一行，将亮度绘制为位置的函数  $f(x)$ ：

问题：有噪声时——边缘在哪里？

噪声的影响：

- 噪声在每个像素产生小的随机波动
- 导数对噪声高度敏感——微小波动被放大
- 真实边缘的导数值可能被噪声淹没

解决方案：先平滑（滤波），再求导。

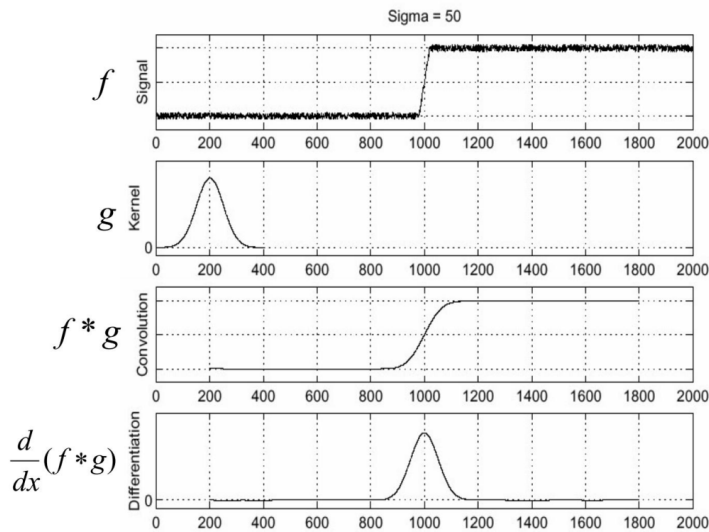
（图片来源：S. Seitz）

学习补充：

- 先平滑再求导是边缘检测里最重要的稳健化步骤之一。
- 原因很简单：求导会放大高频分量，而噪声本身就是高频。



## Solution: smooth first



- To find edges, look for peaks in  $\frac{d}{dx}(f*g)$

Source: S. Seitz

### 方案：先平滑，再求导

流程：

- ① 对信号  $f$  用高斯滤波器  $g$  平滑  $\rightarrow f * g$
- ② 对平滑后的信号求导  $\rightarrow d/dx(f * g)$
- ③ 寻找导数的峰值  $\rightarrow$  峰值位置 = 边缘位置

效果：

- 不滤波直接求导  $\rightarrow$  噪声淹没边缘
- 高斯平滑后求导  $\rightarrow$  清晰的边缘检测

关键洞察：平滑消除了高频噪声，同时保留了真正的边缘信号。

(图片来源：S. Seitz)

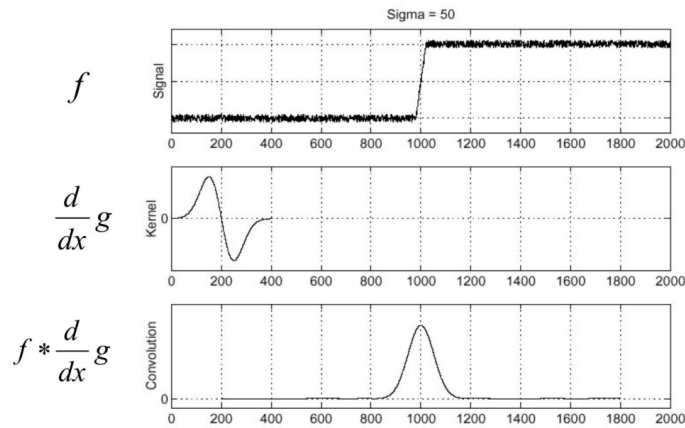
学习补充：

- 先平滑再求导是边缘检测里最重要的稳健化步骤之一。
- 原因很简单：求导会放大高频分量，而噪声本身就是高频。

## Derivative theorem of convolution

- Differentiation is convolution, and convolution is associative:

- This saves us one operation:  $\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$



Source: S. Seitz

## 卷积微分定理（Derivative Theorem）

关键数学性质：微分是卷积，而卷积满足结合律。

$$d/dx(f * g) = f * (d/dxg)$$

实践意义——节省一次运算：

- 不需要：先平滑 → 再求导（两次操作）
- 可以：直接用高斯导数滤波器对原图做一次卷积

结果完全相同，但计算效率更高。

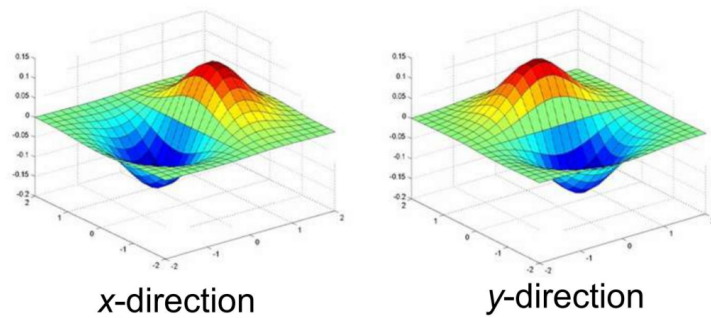
（图片来源：S. Seitz）

学习补充：

- 先对图像做高斯平滑，再求导，等价于直接使用高斯导数核。
- 这样可以把“平滑”和“求导”合并成一步，同时保留边缘、压制噪声。

## Derivative of Gaussian filter

---



Are these filters separable?

Source: S. Lazebnik

---

### 高斯导数滤波器

对高斯函数  $G(x, y)$  求偏导得到两个滤波器:

- $\partial G / \partial x$ : x方向高斯导数 → 检测垂直边缘
- $\partial G / \partial y$ : y方向高斯导数 → 检测水平边缘

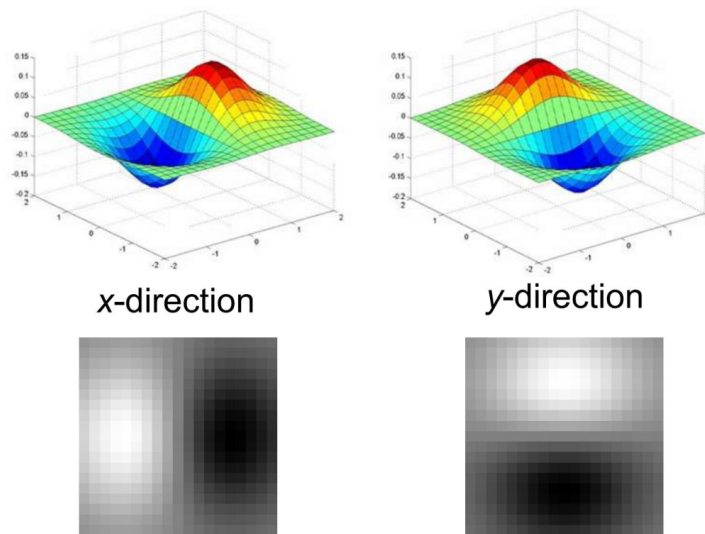
可分离性: 高斯导数滤波器是可分离的——可以分解为两个一维滤波器的级联, 大幅降低计算复杂度。

(图片来源: S. Lazebnik)

学习补充:

- 先对图像做高斯平滑, 再求导, 等价于直接使用高斯导数核。
  - 这样可以把“平滑”和“求导”合并成一步, 同时保留边缘、压制噪声。
-

## Derivative of Gaussian filter



Which one finds horizontal/vertical edges?

Source: S. Lazebnik

### 高斯导数：x方向 vs y方向

判断规则：

- x方向的高斯导数 ( $\partial G / \partial x$ ) : 对水平方向的变化敏感  
→ 检测垂直走向的边缘
- y方向的高斯导数 ( $\partial G / \partial y$ ) : 对垂直方向的变化敏感  
→ 检测水平走向的边缘

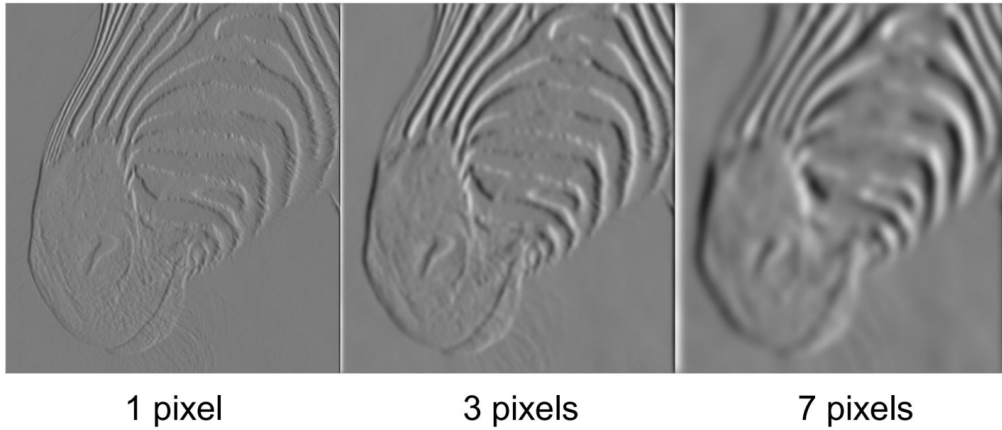
记忆技巧：导数的方向 = 边缘的法线方向，垂直于边缘本身。

(图片来源: S. Lazebnik)

学习补充：

- 先对图像做高斯平滑，再求导，等价于直接使用高斯导数核。
- 这样可以把“平滑”和“求导”合并成一步，同时保留边缘、压制噪声。

## Scale of Gaussian derivative filter



Smoothed derivative removes noise, but blurs edge.  
Also finds edges at different “scales”

Source: D. Forsyth

## 高斯导数滤波器的尺度选择

不同  $\sigma$  (标准差) 对应不同检测尺度:

$\sigma = 1px$  检测细小边缘精确定位对噪声敏感  $\sigma = 3px$  检测中等边缘平衡取舍常用默认值  
 $\sigma = 7px$  检测粗大边缘鲁棒性好定位精度降低

核心矛盾: 小尺度  $\rightarrow$  定位准但噪声敏感; 大尺度  $\rightarrow$  鲁棒但边缘模糊。

(图片来源: D. Forsyth)

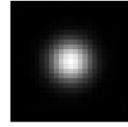
学习补充:

- 多尺度的核心是: 同一个结构在不同尺度下的响应不一样, 不能只看单一尺寸。
- 对 blob 来说, 关键不是“有没有响应”, 而是“在哪个尺度上响应最强”。

## Review: Smoothing vs. derivative filters

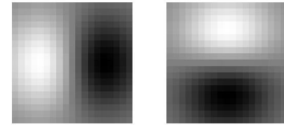
### Smoothing filters

- Gaussian: remove “high-frequency” components; “low-pass” filter
- Can the values of a smoothing filter be negative?
- What should the values sum to?
  - **One:** constant regions are not affected by the filter



### Derivative filters

- Derivatives of Gaussian
- Can the values of a derivative filter be negative?
- What should the values sum to?
- High absolute value at points of high contrast



Source: S. Lazebnik

## 平滑滤波器 vs 导数滤波器

平滑滤波器（高斯）：

- 功能：去除高频成分 → “低通”滤波器
- 值能否为负？不能——平滑是加权平均，负值无意义
- 值之和 = 1——保证恒定亮度区域不受影响

导数滤波器（高斯导数）：

- 功能：检测亮度变化
- 值能否为负？能，必须有——正负交替才能检测变化
- 值之和 = 0——恒定区域没有变化，响应为零
- 在对比度高的位置产生高绝对值响应

(图片来源: S. Lazebnik)

学习补充：

- 多尺度的核心是：同一个结构在不同尺度下的响应不一样，不能只看单一尺寸。
- 对 blob 来说，关键不是“有没有响应”，而是“在哪个尺度上响应最强”。

## The Canny edge detector

---



original image

---

### Canny 边缘检测器

*J. Canny*(1986)提出的经典算法，至今仍是最广泛使用的边缘检测器。

三条最优准则：

- ① 好的检测率——不漏检真实边缘，不误报
- ② 好的定位——检测到的边缘尽可能接近真实边缘位置
- ③ 单一响应——每个边缘只被标记一次

下面逐一介绍 Canny 检测器的四个核心步骤。

学习补充：

- Canny 不是简单阈值法，而是一条完整流水线：平滑、求梯度、细化、连接。
  - 它想要的是“细、稳、连贯”的边缘，而不是一堆粗糙的高响应点。
-

## The Canny edge detector

---



norm of the gradient  $\|\nabla f\| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$

Source: S. Lazebnik

---

### Canny 步骤①：计算梯度幅值

用高斯导数滤波器对图像滤波，计算每个像素的梯度幅值：

$$\|\nabla f\| = \sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2}$$

结果特征：

- 边缘区域 → 梯度幅值大（亮）
- 平坦区域 → 梯度幅值小（暗）
- 边缘线条较粗 → 需要后续细化处理

此时得到的是一张“边缘强度图”，边缘呈现为宽的亮带。

学习补充：

- Canny 不是简单阈值法，而是一条完整流水线：平滑、求梯度、细化、连接。
  - 它想要的是“细、稳、连贯”的边缘，而不是一堆粗糙的高响应点。
-



## The Canny edge detector

---



thresholding

Source: S. Lazebnik

---

### Canny 步骤②：阈值化

对梯度幅值图像应用阈值，区分边缘和非边缘：

- 高阈值 → 只保留最确定的强边缘
- 低阈值 → 保留更多但引入噪声

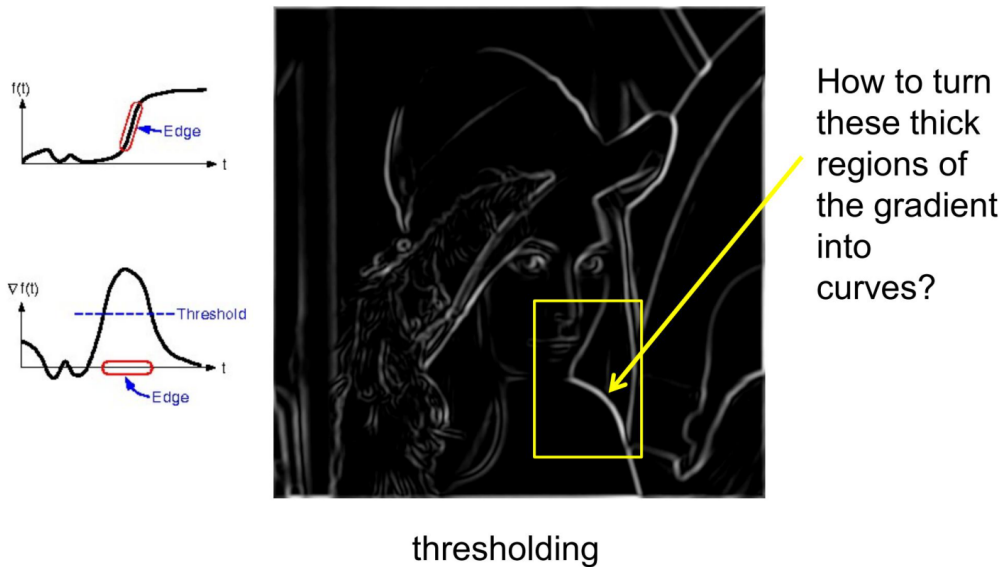
问题：阈值化后的边缘仍然是粗的带（多像素宽），不是我们想要的细曲线。

下一步需要：将粗带细化为单像素宽度的曲线。

学习补充：

- Canny 不是简单阈值法，而是一条完整流水线：平滑、求梯度、细化、连接。
  - 它想要的是“细、稳、连贯”的边缘，而不是一堆粗糙的高响应点。
-

# The Canny edge detector



Source: S. Lazebnik

## Canny 需要解决的两个问题

阈值化后的梯度幅值图存在两个关键问题：

问题 ①：边缘太粗

边缘呈现为多像素宽的"脊"，需要细化为单像素曲线。

→ 解决方案：非极大值抑制（NMS）

问题 ②：边缘断裂

低对比度边缘被阈值切断，强边缘之间失去连接。

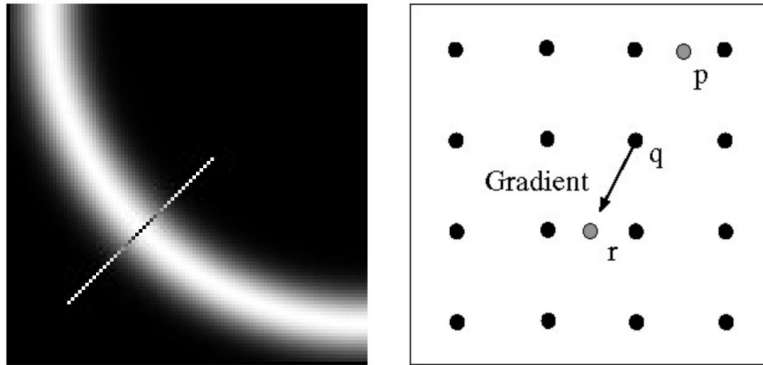
→ 解决方案：滞后阈值（Hysteresis）

接下来分别介绍这两个关键技术。

学习补充：

- Canny 不是简单阈值法，而是一条完整流水线：平滑、求梯度、细化、连接。
- 它想要的是“细、稳、连贯”的边缘，而不是一堆粗糙的高响应点。

## Non-maximum suppression



Check if pixel is local maximum along gradient direction, select single max across width of the edge

Source: S. Lazebnik

### 非极大值抑制（NMS）

目标：将粗边缘细化为单像素宽度。

算法流程：

- ① 对每个像素，沿其梯度方向检查相邻像素
- ② 如果当前像素的梯度幅值 不是梯度方向上的局部最大值
- ③ 则将其抑制（置为 0）
- ④ 只保留该方向上的最大响应值

结果：原来多像素宽的边缘被压缩为单像素宽的细线。

（图片来源：S. Lazebnik）

学习补充：

- 非极大值抑制沿梯度方向做比较，只保留局部峰值。
- 目的不是找“更大的数”，而是把边缘细化成单像素宽。

## The Canny edge detector

---



thinning

(non-maximum suppression)

Source: S. Lazebnik

---

### Canny 完整处理流程

原始图像

↓ 高斯导数滤波

梯度幅值图（边缘粗，含噪声）

↓ 非极大值抑制（NMS）

细化后的边缘（单像素宽，仍有假边缘）

↓ 滞后阈值

最终边缘图（清晰、连续、单像素宽）

NMS 解决了“粗”的问题，但噪声引起的假边缘仍需阈值处理。

（图片来源：S. Lazebnik）

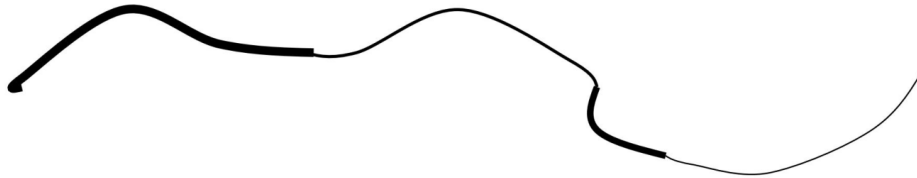
学习补充：

- 非极大值抑制沿梯度方向做比较，只保留局部峰值。
- 目的不是找“更大的数”，而是把边缘细化成单像素宽。

## Hysteresis thresholding

---

Use a high threshold to start edge curves, and a low threshold to continue them.



Source: S. Seitz

---

### 滞后阈值（Hysteresis Thresholding）

使用两个阈值而不是一个：

- 高阈值  $T_{high}$ ：用于启动（seed）边缘曲线  
→ 只有梯度幅值  $> T_{high}$  的点被标记为“确定边缘”
- 低阈值  $T_{low}$ ：用于延续（extend）边缘曲线  
→ 从确定边缘点出发，沿边缘方向追踪  
→ 只要梯度幅值  $> T_{low}$  就继续延伸

效果：强边缘完整保留，连接在强边缘上的弱边缘被保留，孤立的弱边缘（噪声）被丢弃。

（图片来源：S. Seitz）

学习补充：

- 双阈值的作用是把“可信边缘”和“可能边缘”分开。
  - 低阈值像是候选池，高阈值像是锚点；只有连到锚点的弱响应才会被保留。
-

## Hysteresis thresholding

---



original image



high threshold  
(strong edges)



low threshold  
(weak edges)



hysteresis threshold

Source: L. Fei-Fei

---

### 滞后阈值——三种阈值效果对比

同一张原始图像，三种阈值策略的结果：

- 仅用高阈值：只有强边缘——可能不完整、断裂
- 仅用低阈值：边缘完整——但包含大量噪声
- 滞后阈值（高+低）：边缘完整且噪声少——最佳效果

这是 Canny 检测器超越简单边缘检测方法的关键设计。

(图片来源：L. Fei-Fei)

学习补充：

- 双阈值的作用是把“可信边缘”和“可能边缘”分开。
  - 低阈值像是候选池，高阈值像是锚点；只有连到锚点的弱响应才会被保留。
-

## Recap: Canny edge detector

1. Filter image with derivative of Gaussian
2. Find magnitude and orientation of gradient
3. **Non-maximum suppression:**
  - Thin wide “ridges” down to single pixel width
4. **Linking and thresholding (hysteresis):**
  - Define two thresholds: low and high
  - Use the high threshold to start edge curves and the low threshold to continue them

MATLAB: `edge(image, 'canny');`

J. Canny, [A Computational Approach To Edge Detection](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Source: S. Lazebnik

## Canny 边缘检测器——完整算法总结

步骤 1: 高斯导数滤波

对图像使用高斯导数滤波器 → 同时完成平滑和微分

步骤 2: 计算梯度幅值和方向

$$||\nabla f|| = \sqrt{(\partial f / \partial x)^2 + (\partial f / \partial y)^2} \quad \theta = \tan^{-1}(\partial y / \partial x)$$

步骤 3: 非极大值抑制

沿梯度方向只保留局部最大值 → 宽脊变为单像素

步骤 4: 滞后阈值

高阈值启动边缘 + 低阈值延续边缘 → 连续且干净的边缘图

MATLAB: `edge(image, 'canny')`

OpenCV: `cv2.Canny(image, low, high)`

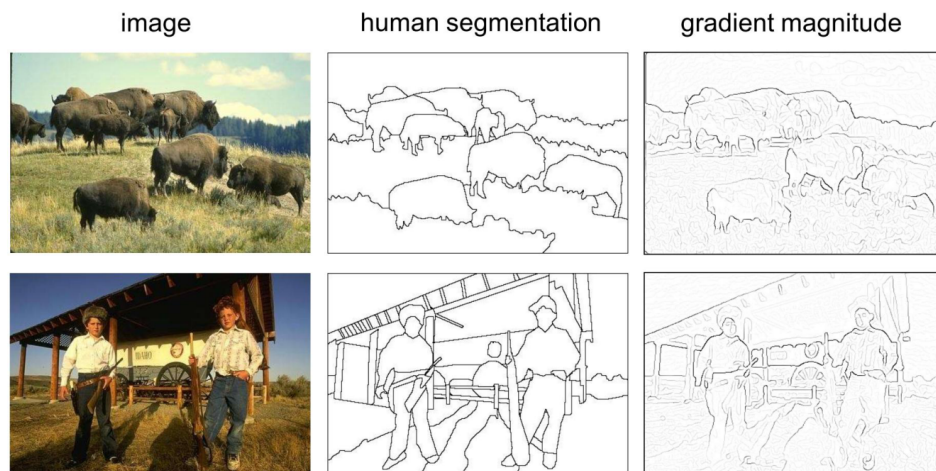
参考文献: J. Canny, "A Computational Approach To Edge Detection", IEEE TPAMI, 8:679-714, 1986.

学习补充:

- 这页是在总结 Canny 的思想: 先把边缘找出来, 再把边缘变细, 再把断裂边缘连起来。
- 这比单纯看阈值稳定得多, 也更符合人眼对边缘的理解。

## Edge detection is just the beginning...

---



Berkeley segmentation database:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/segbench/>

Source: S. Lazechnik

---

## 边缘检测只是起点.....

边缘检测是通往更高级视觉理解的第一步:

原始图像 → 人工标注 → 分割结果 → 边缘检测

边缘信息是以下任务的基础输入:

- 图像分割——基于边缘的轮廓分割
- 物体识别——形状匹配与轮廓分析
- 场景理解——从边缘到语义

Berkeley 分割数据库 (BSDS500) :

标准评估基准:

<http://www.eecs.berkeley.edu/Research/Projects/CS/vision/grouping/>

(图片来源: S. Lazechnik)

---





---

## 本章小结

1. 边缘 = 图像亮度函数的快速变化位置
2. 图像梯度  $\rightarrow$  偏导数  $\rightarrow$  梯度幅值 = 边缘强度
3. 噪声问题  $\rightarrow$  高斯平滑后再求导
4. 卷积微分定理  $\rightarrow$  高斯导数滤波器: 一次卷积 = 平滑 + 求导
5. 离散导数算子  $\rightarrow$  *Sobel*, *Prewitt*, *Roberts*
6. Canny 边缘检测器 (四步法):  
高斯导数滤波  $\rightarrow$  梯度幅值/方向  $\rightarrow$  *NMS*  $\rightarrow$  滞后阈值

下一章预告: 拟合——将检测到的边缘点拟合成直线、圆等几何形状。

---